

Quiz 3

Tick the statements that are true and leave unticked those that are false.

Each statement is worth 1 point for a correct answer.

Statements with a * are bonus.

Your name: _____

Which of the following statements are correct?

- ☒ A Turing machine is a mathematical model of the informal notion of “algorithm.”
- ☐ Before Turing, no one ever thought of algorithms.
- ☐ The Turing machine was designed to solve the Turing test.
- ☒ Euclid’s algorithm for finding the greatest common divisor of two numbers can be executed by a Turing machine.
- ☒ Any Turing machine with a finite tape alphabet Γ can be simulated by a Turing machine with the binary tape alphabet $\{0, 1\}$.
- ☒ If a Turing machine ever repeats the exact same configuration (state, tape content, and head position) during a computation, it will never halt.
- ☐ The Church–Turing Thesis is a mathematical theorem independently proved by Alonzo Church and Alan Turing in 1936.
- ☐ Every Turing machine halts on at least one input.
- ☒ For every program in C, there is a Turing machine that computes exactly the same function.
- ☐ In C++, one can write a program that no Turing machine is equivalent to, because machines are inherently procedural and cannot work in the object-oriented paradigm.
- ☐ *A Turing machine halts on every input word if and only if in its transition graph there is no loop on the path from the initial state to the halting state.
- ☐ *Turing machines working on a two-dimensional plane (instead of a one-dimensional tape) are more powerful (i.e. can compute more functions) than standard Turing machines.

If you were a Turing machine, which function would you compute?

Explanations

- ⊠ *A Turing machine is a mathematical model of the informal notion of “algorithm.”* Giving such a definition was precisely Turing’s aim.
- *Before Turing, no one ever thought of algorithms.* Algorithms have been known and used for millennia (e.g. Euclid’s algorithm for finding the greatest common divisor in c. 300 BC).
- *The Turing machine was designed to solve the Turing test.* The Turing test, concerning whether machines can exhibit human-like intelligence, was proposed by Turing much later.
- ⊠ *Euclid’s algorithm for finding the greatest common divisor of two numbers can be executed by a Turing machine.* Euclid’s algorithm performs elementary arithmetic operations (subtractions or divisions with remainder), all of which can easily be programmed on a Turing machine.
- ⊠ *Any Turing machine with a finite tape alphabet Γ can be simulated by a Turing machine with the binary tape alphabet $\{0, 1\}$.* By encoding each symbol of Γ as a fixed-length block of k bits (where $2^k \geq |\Gamma|$), a binary machine can read, write, and process each original cell block by block using finitely many internal states to track prefixes.
- ⊠ *If a Turing machine ever repeats the exact same configuration (state, tape content, and head position) during a computation, it will never halt.* Having the exact same configuration at step $t_2 > t_1$ means that the machine will repeat the exact same sequence of transitions from t_1 to t_2 indefinitely, entering an infinite periodic loop.
- *The Church–Turing Thesis is a mathematical theorem independently proved by Alonzo Church and Alan Turing in 1936.* The Church–Turing Thesis asserts that the intuitive notion of an algorithm coincides with computability by a Turing machine. It is an empirical hypothesis linking an informal notion to the formal one, so it cannot be proved mathematically.
- *Every Turing machine halts on at least one input.* A machine that moves right and never changes its state regardless of what it reads never halts on any input.
- ⊠ *For every program in C , there is a Turing machine that computes exactly the same function.* Any physical computer can be mechanically simulated step by step by a Turing machine. This can also be seen as a consequence of the Church–Turing Thesis.
- *In $C++$, one can write a program that no Turing machine is equivalent to, because machines are inherently procedural and cannot work in the object-oriented paradigm.* Programming paradigms (procedural, object-oriented, functional) are human-facing abstractions. $C++$ programs compile down to machine instructions and can compute only the same functions that a Turing machine can compute.
- **A Turing machine halts on every input word if and only if in its transition graph there is no loop on the path from the initial state to the halting state.* Existence of loops in a graph is decidable, and thus cannot correspond to the property ‘ M halts on every input’, which is semantic and hence undecidable by Rice’s theorem.
- **Turing machines working on a two-dimensional plane (instead of a one-dimensional tape) are more powerful (i.e. can compute more functions) than standard Turing machines.* A two-dimensional Turing machine is easy to simulate with a C program (say), which, as noted above, can in turn be simulated by a standard Turing machine.